

Phase 5: Enhanced Instruction Set Implementation

Objective

Expand the CPU instruction set to include memory operations, register-to-register transfers, and stack operations for complete programming capability.

Component BOM

Additional Components

Qty	Component	Function
4	SN74ALS573BN	General-purpose register file (4x 8-bit registers)
2	SN74ALS652NT	Registered transceivers (register multiplexing)
1	74LS138	3-to-8 decoder (register select)
2	74F157	Quad 2-to-1 multiplexers (data source select)
1	74F163APC	4-bit counter (stack pointer)
1	74LS245	Bidirectional transceiver (memory interface)
2	74F08	Additional AND gates (control logic)
1	74F32DC	OR gates (signal combining)
8	0.1μF capacitors	Additional decoupling
16	DIP switches	Register select testing
-	Additional breadboard space	Register file construction

Expanded Architecture

Register File

- R0: General-purpose register (accumulator)
- R1: General-purpose register
- R2: General-purpose register
- R3: General-purpose register
- SP: Stack pointer (74F163APC)
- PC: Program counter (existing 74LS190N chain)
- FLAGS: Zero, Carry, Overflow flags (existing 74LS74)

Memory Map Enhancement

0x0000-0x7FFF: Program memory (32KB)
0x8000-0x8FFF: Data memory (4KB)
0x9000-0x9FFF: Stack space (4KB)
0xA000-0xAFFF: I/O space (4KB, future peripherals)
0xB000-0xFFFF: Reserved (20KB)

Complete Instruction Set

Data Transfer Instructions

Opcode	Mnemonic	Description	Cycles
0000	NOP	No operation	1
0001	LD Rn,#imm	Load immediate into register	2
0010	LD Rn,addr	Load from memory into register	3
0011	ST Rn,addr	Store register to memory	3
0100	MOV Rn,Rm	Move register to register	1

Arithmetic Instructions

Opcode	Mnemonic	Description	Cycles
0101	ADD Rn,Rm	Add registers	1
0110	SUB Rn,Rm	Subtract registers	1
0111	INC Rn	Increment register	1
1000	DEC Rn	Decrement register	1

Logical Instructions

Opcode	Mnemonic	Description	Cycles
1001	AND Rn,Rm	Logical AND	1
1010	OR Rn,Rm	Logical OR	1
1011	XOR Rn,Rm	Logical XOR	1
1100	NOT Rn	Logical NOT	1

Control Flow Instructions

Opcode	Mnemonic	Description	Cycles
1101	JMP addr	Unconditional jump	3
1110	JZ addr	Jump if zero	3
1111	JC addr	Jump if carry	3

Extended Instructions (Future)

CALL addr - Subroutine call with stack

RET - Return from subroutine

PUSH Rn - Push register to stack

POP Rn - Pop stack to register

CMP Rn,Rm - Compare registers

Instruction Encoding

Register Encoding (3 bits)

000 = R0 (accumulator)

001 = R1

010 = R2

011 = R3

100 = SP (stack pointer)

101 = PC (program counter, special cases)

110 = FLAGS (read-only)

111 = Reserved

Multi-Byte Instruction Format

Single-byte instructions:

[7:4] = Opcode, [3:1] = Dest Reg, [0] = Src Reg/Mode

Two-byte instructions:

Byte 0: [7:4] = Opcode, [3:0] = Register/Mode

Byte 1: Immediate data

Three-byte instructions:

Byte 0: [7:4] = Opcode, [3:0] = Mode

Byte 1: Address high byte

Byte 2: Address low byte

Circuit Implementation

Register File Architecture

Register Select Logic:

74LS138 decoder → Register enable signals

3-bit register address → Decoder inputs

Read/Write control → Decoder enable

Data Routing:

SN74ALS652NT transceivers → Register bank access

74F157 multiplexers → Source register selection

ALU connection → Register arithmetic operations

Memory Interface Enhancement

Address Generation:

Register indirect: Register contents → Address bus

Immediate addressing: Instruction data → Address bus

Stack operations: SP register → Address bus

Data Path Control:

74LS245 transceiver → Bidirectional memory access

Read/Write timing → Memory control signals

Bus arbitration → Prevent contention

Stack Implementation

Stack Pointer (SP):

74F163APC counter → Stack address generation

Auto-increment/decrement → PUSH/POP operations

Stack bounds checking → Overflow/underflow detection

Stack Operations:

PUSH: Write data to [SP], increment SP

POP: Decrement SP, read data from [SP]

CALL: PUSH PC, JMP addr

RET: POP PC

Build Steps

1. Register File Construction

- Install four SN74ALS573BN latches for general registers
- Add 74LS138 decoder for register selection
- Wire register enable and data path connections
- Test manual register read/write operations

2. Enhanced Instruction Decode

- Expand MC74F521N comparator logic for new opcodes
- Add control signal generation for register operations
- Implement multi-byte instruction handling
- Test instruction recognition and control generation

3. ALU-Register Integration

- Connect register file to ALU input multiplexers
- Add register-to-register operation capability
- Implement ALU result storage to any register
- Test computational instructions with register operands

4. Memory Interface Enhancement

- Add 74LS245 bidirectional transceiver for memory access
- Implement register indirect addressing
- Add load/store instruction execution
- Test memory read/write with register addressing

5. Stack Operations

- Install 74F163APC stack pointer counter
- Add stack memory addressing logic
- Implement PUSH/POP instruction execution
- Test subroutine call/return capability

Test Procedures

1. Register File Test

Objective: Verify register read/write operations

Manual Test:

1. Select register R0 (address 000)
2. Write test data 0xAA via data bus
3. Pulse register write enable
4. Read back data, verify 0xAA
5. Repeat for all registers R0-R3

Automated Test Program:

```
0x0000: LD R0,#0x55 ; Load 0x55 into R0
0x0002: LD R1,#0xAA ; Load 0xAA into R1
0x0004: MOV R2,R0 ; Copy R0 to R2
0x0005: MOV R3,R1 ; Copy R1 to R3
0x0006: NOP ; Display current state
```

2. Register Arithmetic Test

Objective: Verify ALU operations with register operands

Test Program:

```
0x0000: LD R0,#0x0F ; Load 15 into R0
0x0002: LD R1,#0x01 ; Load 1 into R1
0x0004: ADD R0,R1 ; R0 = R0 + R1 (result: 16)
0x0005: LD R2,#0x05 ; Load 5 into R2
0x0007: SUB R0,R2 ; R0 = R0 - R2 (result: 11)
0x0008: NOP ; Display result
```

Expected Result: R0 contains 0x0B (11 decimal)

3. Memory Operations Test

Objective: Verify load/store instructions

Test Program:

```
0x0000: LD R0,#0xFF ; Load test data
0x0002: ST R0,0x8000 ; Store to memory location 0x8000
0x0005: LD R1,#0x00 ; Clear R1
0x0007: LD R1,0x8000 ; Load from memory to R1
0x000A: NOP ; R1 should now contain 0xFF
```

4. Stack Operations Test

Objective: Verify PUSH/POP functionality

Test Program:

```
0x0000: LD R0,#0x12 ; Load test data
0x0002: LD R1,#0x34 ; Load test data
0x0004: PUSH R0 ; Push R0 to stack
0x0005: PUSH R1 ; Push R1 to stack
0x0006: LD R0,#0x00 ; Clear R0
0x0008: LD R1,#0x00 ; Clear R1
0x000A: POP R1 ; Pop to R1 (should get 0x34)
0x000B: POP R0 ; Pop to R0 (should get 0x12)
0x000C: NOP ; Verify restoration
```

5. Complex Program Test

Objective: Execute a meaningful program using multiple instruction types

Fibonacci Sequence Generator:

```
; Calculate Fibonacci:  $F(n) = F(n-1) + F(n-2)$ 
0x0000: LD R0,#0x00 ; F(0) = 0
0x0002: LD R1,#0x01 ; F(1) = 1
0x0004: MOV R2,R0 ; Store F(n-2)
0x0005: MOV R3,R1 ; Store F(n-1)
0x0006: ADD R0,R1 ; R0 = F(n-2) + F(n-1)
0x0007: MOV R2,R3 ; Update F(n-2)
0x0008: MOV R3,R0 ; Update F(n-1)
0x0009: JMP 0x0006 ; Loop for next Fibonacci number
```

Expected Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

Logic Analyzer Validation

Extended Channel Assignment

Channels	Signal	Purpose
0-15	Address bus	Memory access verification
16-23	Data bus	Instruction and data flow
24-26	Register select	Register file addressing
27-30	Register enables	Read/write control
31-38	Register R0 output	Register content verification
39-46	Register R1 output	Register content verification
47-54	ALU output	Computation verification
55-58	Stack pointer	Stack operation tracking
59	Memory read enable	Memory access timing
60	Memory write enable	Memory access timing
61-63	Control state signals	Instruction cycle phases

Success Criteria

- ☐ All registers read/write correctly with proper isolation
- ☐ Register-to-register operations execute accurately
- ☐ Memory load/store instructions access correct addresses
- ☐ ALU operations work with register operands
- ☐ Stack operations maintain proper LIFO behavior
- ☐ Multi-byte instructions decode and execute correctly
- ☐ Complex programs execute without errors
- ☐ All instruction cycle timing meets requirements

Performance Validation

Instruction Throughput

- Measure cycles per instruction for each type
- Determine maximum sustainable execution rate
- Verify timing margins for back-to-back instructions

Memory Access Performance

- Test sequential vs. random memory access patterns
- Verify SRAM timing requirements met
- Measure memory bandwidth utilization

Register File Performance

- Test register access speed
- Verify register multiplexer timing
- Confirm no timing races in register operations

Integration Notes for Phase 6

- Document complete instruction set timing
- Identify I/O instruction requirements
- Plan peripheral interface architecture
- Note memory map utilization
- Record performance characteristics

Documentation Deliverables

- Complete instruction set reference manual
- Programming examples and test programs
- Register file architecture documentation
- Memory interface specifications
- Performance benchmark results
- Integration schematic with complete CPU architecture